

Vue3 插值表达式的用法与注意事项

Vue3 的模板语法中，**插值表达式（Mustache 语法，`{{ }}`**）是最基础、最核心的数据绑定方式，用于将组件逻辑层的数据渲染到模板视图层。以下从**基础用法**、**进阶场景**、**注意事项**三方面详细讲解，结合 Vue3 `<script setup>` 语法糖示例说明：

一、基础用法：绑定基础数据

插值表达式的核心是「将响应式数据/普通变量渲染到 DOM」，其中**使用ref定义响应式数据变量**是 Vue3 中最基础的响应式方案，尤其适用于基础数据类型（字符串、数字、布尔值等）。ref 会包装数据形成响应式对象，通过 ``.value`` 访问和修改内部值，插值表达式可直接绑定 ref 变量（无需手动写 ``.value``），支持简单的表达式计算。

1. 绑定响应式变量（核心：基于ref实现）

Code block

```
1  <template>
2    <!-- 绑定字符串 -->
3    <div>用户名: {{ username }}</div>
4    <!-- 绑定数字 -->
5    <div>年龄: {{ age }}</div>
6    <!-- 绑定布尔值（需结合逻辑展示） -->
7    <div>是否登录: {{ isLogin ? "已登录" : "未登录" }}</div>
8  </template>
9
10 <script setup>
11 import { ref } from "vue";
12 // 响应式数据
13 const username = ref("张三");
14 const age = ref(25);
15 const isLogin = ref(true);
16 </script>
```

2. 绑定普通变量（非响应式）

插值表达式也支持非响应式变量，但变量更新后视图不会自动刷新（需配合响应式 API 才会触发更新）：

Code block

```
1  <template>
2    <div>普通变量: {{ normalText }}</div>
```

```
3   </template>
4
5   <script setup>
6     // 非响应式变量
7     const normalText = "这是普通文本";
8   </script>
```

二、进阶用法：支持的表达式类型

Vue3 插值表达式支持**单个表达式**（不支持语句/流程控制），常见场景如下：

表达式类型	示例代码
算术运算	<code>{{ num1 + num2 }}</code> 、 <code>{{ price * count }}</code> 、 <code>{{ 10 / 2 }}</code>
字符串拼接	<code>{{ "Hello, " + username }}</code> 、 <code>{{ `Hello, \${username}` }}</code> (模板字符串)
三元运算符	<code>{{ score >= 60 ? "及格" : "不及格" }}</code>
数组/对象取值	<code>{{ userInfo.name }}</code> 、 <code>{{ list[0] }}</code> 、 <code>{{ obj["age"] }}</code>
调用方法	<code>{{ formatTime(createTime) }}</code> （调用组件内定义的方法）
计算属性	<code>{{ reversedName }}</code> (绑定计算属性，推荐优先使用)
布尔判断	<code>{{ isShow && "显示内容" }}</code> （短路运算）

完整示例：

```
Code block
1   <template>
```

```
2    <view class="demo">
3      <!-- 算术运算 -->
4      <div>总价: {{ price * count }} 元</div>
5      <!-- 字符串拼接 -->
6      <div>欢迎语: {{ `你好, ${username}! ` }}</div>
7      <!-- 数组/对象取值 -->
8      <div>用户爱好: {{ hobbies[1] }}</div>
9      <div>用户邮箱: {{ userInfo.email }}</div>
10     <!-- 调用方法 -->
11     <div>格式化时间: {{ formatDate(new Date()) }}</div>
12     <!-- 计算属性 -->
13     <div>反转用户名: {{ reversedName }}</div>
14   </view>
15 </template>
16
17 <script setup>
18 import { ref, computed } from "vue";
19
20 // 基础数据
21 const price = ref(99);
22 const count = ref(3);
23 const username = ref("李四");
24 const hobbies = ref(["看书", "运动", "听歌"]);
25 const userInfo = ref({
26   email: "lisi@example.com",
27   age: 28
28 });
29
30 // 自定义方法
31 const formatDate = (date) => {
32   return date.toLocaleDateString(); // 格式化日期为本地格式
33 };
34
35 // 计算属性（推荐：缓存结果，性能更优）
36 const reversedName = computed(() => {
37   return username.value.split("").reverse().join("");
38 });
39 </script>
```

三、注意事项：禁止/不推荐的用法

插值表达式有明确的语法限制，以下场景**不支持或不推荐**：

1. 禁止使用语句/流程控制

插值表达式仅支持「单个表达式」，不支持 `if`、`for`、`while` 等语句，也不支持变量赋值：

```

1 <template>
2   <!-- ❌ 错误：if 语句不允许 -->
3   <div>{{ if (age > 18) { return "成年" } }}</div>
4   <!-- ❌ 错误：赋值语句不允许 -->
5   <div>{{ age = 30 }}</div>
6   <!-- ❌ 错误：多个表达式（用逗号分隔） -->
7   <div>{{ a = 1, b = 2 }}</div>
8
9   <!-- ✅ 正确：用三元运算符替代 if -->
10  <div>{{ age > 18 ? "成年" : "未成年" }}</div>
11 </template>

```

2. 不推荐复杂逻辑

插值表达式内应保持「简单计算」，复杂逻辑（如多层判断、复杂运算）建议抽离为**计算属性或方法**，提升可读性和可维护性：

Code block

```

1 <template>
2   <!-- ❌ 不推荐：复杂逻辑直接写在插值中 -->
3   <div>{{ score > 90 ? "优秀" : score > 80 ? "良好" : score > 60 ? "及格" : "不及格" }}</div>
4
5   <!-- ✅ 推荐：抽离为计算属性 -->
6   <div>{{ scoreLevel }}</div>
7 </template>
8
9 <script setup>
10 import { ref, computed } from "vue";
11 const score = ref(85);
12 const scoreLevel = computed(() => {
13   if (score.value > 90) return "优秀";
14   if (score.value > 80) return "良好";
15   if (score.value > 60) return "及格";
16   return "不及格";
17 });
18 </script>

```

3. 避免直接调用异步方法

插值表达式中调用异步方法（如接口请求）会导致重复执行，且无法正确渲染结果，需在生命周期/事件中执行异步逻辑，结果存入响应式变量后再渲染：

Code block

```

1  <template>
2    <!-- ❌ 错误：异步方法直接插值 -->
3    <div>{{ fetchData() }}</div>
4
5    <!-- ✅ 正确：先请求数据，再渲染 -->
6    <div>{{ asyncData }}</div>
7  </template>
8
9  <script setup>
10 import { ref, onMounted } from "vue";
11 const asyncData = ref("");
12
13 // 异步请求方法
14 const fetchData = async () => {
15   const res = await fetch("https://api.example.com/data");
16   const data = await res.json();
17   asyncData.value = data.name;
18 };
19
20 // 挂载后执行请求
21 onMounted(() => {
22   fetchData();
23 });
24 </script>

```

4. 特殊数据渲染处理

- **HTML 内容：**插值表达式会将内容作为纯文本渲染（转义 HTML 标签），若需渲染 HTML，需使用 `v-html` 指令：

Code block

```

1  <template>
2    <!-- 渲染为纯文本：<span>加粗文本</span> -->
3    <div>{{ htmlText }}</div>
4    <!-- 渲染为 HTML：<span>加粗文本</span>（注意 XSS 风险） -->
5    <div v-html="htmlText"></div>
6  </template>
7
8  <script setup>
9    const htmlText = '<span style="font-weight: bold;">加粗文本</span>';
10 </script>

```

- **null/undefined：**插值表达式遇到 `null` / `undefined` 会渲染为空字符串，无需额外处理：

```
1 <template>
2   <div>{{ nullValue }}</div> <!-- 渲染为空 -->
3   <div>{{ undefinedValue }}</div> <!-- 渲染为空 -->
4 </template>
5
6 <script setup>
7   const nullValue = null;
8   const undefinedValue = undefined;
9 </script>
```

四、Vue3 新增/优化点

1. **多根节点支持**：Vue3 模板允许多个根节点，插值表达式可分散在不同根节点中，无需外层包裹：

Code block

```
1 <template>
2   <div>{{ username }}</div>
3   <div>{{ age }}</div>
4 </template>
```

2. **响应式解构兼容**：配合 `toRefs` 解构响应式对象时，插值表达式可直接使用解构后的变量：

Code block

```
1 <template>
2   <div>{{ name }}</div>
3   <div>{{ age }}</div>
4 </template>
5
6 <script setup>
7   import { reactive, toRefs } from "vue";
8   const user = reactive({ name: "王五", age: 30 });
9   const { name, age } = toRefs(user); // 解构为响应式 ref
10 </script>
```

总结

Vue3 插值表达式的核心规则：

1. 基础用法： `{{ 变量/简单表达式 }}` ，绑定响应式数据实现视图更新；
2. 表达式限制：仅支持「单个表达式」，禁止语句/复杂逻辑；
3. 最佳实践：复杂逻辑抽离为计算属性/方法，HTML 内容用 `v-html` ，异步数据先请求再渲染；
4. 优势：简洁高效，配合 Vue3 响应式 API 可快速实现数据与视图的双向联动。

(注：文档部分内容可能由 AI 生成)